In [1]:
```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

In [2]:
```python
df = pd.read_csv("expected_ctc.csv")
df = df.drop(["IDX","Applicant_ID"],axis=1)
df.head()
```

Out[2]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduation_Spec
0	0	0	NaN	NaN	NaN	NaN	NaN	PG	
1	23	14	HR	Consultant	Analytics	H	HR	Doctorate	
2	21	12	Top Management	Consultant	Training	J	NaN	Doctorate	
3	15	8	Banking	Financial Analyst	Aviation	F	HR	Doctorate	
4	10	5	Sales	Project Manager	Insurance	E	Medical Officer	Grad	

5 rows × 27 columns

In [3]:
```python
df.head(5)
```

Out[3]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduation_Spec
0	0	0	NaN	NaN	NaN	NaN	NaN	PG	
1	23	14	HR	Consultant	Analytics	H	HR	Doctorate	
2	21	12	Top Management	Consultant	Training	J	NaN	Doctorate	
3	15	8	Banking	Financial Analyst	Aviation	F	HR	Doctorate	
4	10	5	Sales	Project Manager	Insurance	E	Medical Officer	Grad	

5 rows × 27 columns

In [4]:
```python
df.tail(5)
```

Out[4]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduatio
24995	18	13	Engineering	Project Manager	Automobile	I	Assistant Manager	PG	
24996	12	8	HR	Others	Analytics	B	Sr.Manager	Under Grad	
24997	22	8	Banking	Head	Insurance	D	Software Developer	Under Grad	
24998	25	8	Marketing	CEO	BFSI	D	Marketing Manager	PG	
24999	8	0	Banking	Consultant	Automobile	P	Sr.Manager	Grad	

5 rows × 27 columns

In [5]:
```python
df.columns
```

Out[5]:
```
Index(['Total_Experience', 'Total_Experience_in_field_applied', 'Department',
       'Role', 'Industry', 'Organization', 'Designation', 'Education',
       'Graduation_Specialization', 'University_Grad',
       'Passing_Year_Of_Graduation', 'PG_Specialization', 'University_PG',
       'Passing_Year_Of_PG', 'PHD_Specialization', 'University_PHD',
       'Passing_Year_Of_PHD', 'Curent_Location', 'Preferred_location',
       'Current_CTC', 'Inhand_Offer', 'Last_Appraisal_Rating',
       'No_Of_Companies_worked', 'Number_of_Publications', 'Certifications',
       'International_degree_any', 'Expected_CTC'],
      dtype='object')
```

In [6]:
```python
df.columns.to_series().groupby(df.dtypes).groups
```

Out[6]:
```
{int64: ['Total_Experience', 'Total_Experience_in_field_applied', 'Current_CTC', 'No_Of_Companies_worked', 'Num
ber_of_Publications', 'Certifications', 'International_degree_any', 'Expected_CTC'], float64: ['Passing_Year_Of
_Graduation', 'Passing_Year_Of_PG', 'Passing_Year_Of_PHD'], object: ['Department', 'Role', 'Industry', 'Organiz
ation', 'Designation', 'Education', 'Graduation_Specialization', 'University_Grad', 'PG_Specialization', 'Unive
rsity_PG', 'PHD_Specialization', 'University_PHD', 'Curent_Location', 'Preferred_location', 'Inhand_Offer', 'La
st_Appraisal_Rating']}
```

In [7]:
```python
df.info()
```

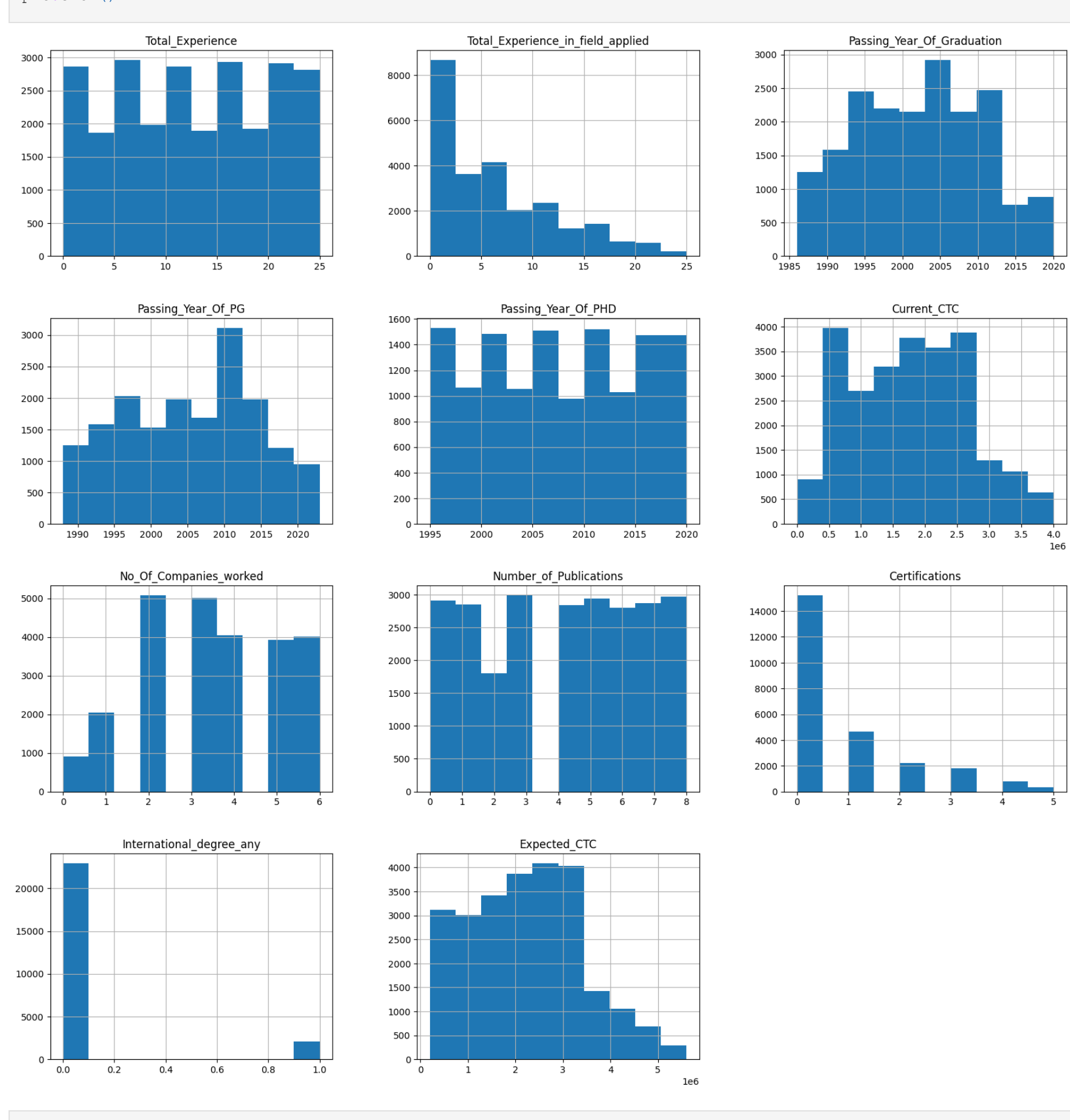
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 27 columns):
 #   Column                             Non-Null Count  Dtype
---  ------                             --------------  -----
 0   Total_Experience                   25000 non-null  int64
 1   Total_Experience_in_field_applied  25000 non-null  int64
 2   Department                         22222 non-null  object
 3   Role                               24037 non-null  object
 4   Industry                           24092 non-null  object
 5   Organization                       24092 non-null  object
 6   Designation                        21871 non-null  object
 7   Education                          25000 non-null  object
 8   Graduation_Specialization          18820 non-null  object
 9   University_Grad                    18820 non-null  object
 10  Passing_Year_Of_Graduation         18820 non-null  float64
 11  PG_Specialization                  17308 non-null  object
 12  University_PG                      17308 non-null  object
 13  Passing_Year_Of_PG                 17308 non-null  float64
 14  PHD_Specialization                 13119 non-null  object
 15  University_PHD                     13119 non-null  object
 16  Passing_Year_Of_PHD                13119 non-null  float64
 17  Curent_Location                    25000 non-null  object
 18  Preferred_location                 25000 non-null  object
 19  Current_CTC                        25000 non-null  int64
 20  Inhand_Offer                       25000 non-null  object
 21  Last_Appraisal_Rating              24092 non-null  object
 22  No_Of_Companies_worked             25000 non-null  int64
 23  Number_of_Publications             25000 non-null  int64
 24  Certifications                     25000 non-null  int64
 25  International_degree_any           25000 non-null  int64
 26  Expected_CTC                       25000 non-null  int64
dtypes: float64(3), int64(8), object(16)
memory usage: 5.1+ MB
```

In [8]:
```python
df.describe()
```

Out[8]:

	Total_Experience	Total_Experience_in_field_applied	Passing_Year_Of_Graduation	Passing_Year_Of_PG	Passing_Year_Of_PHD	Current_CTC
count	25000.000000	25000.000000	18820.000000	17308.000000	13119.000000	2.500000e+04
mean	12.493080	6.258200	2002.193624	2005.153571	2007.396372	1.760945e+06
std	7.471398	5.819513	8.316640	9.022963	7.493601	9.202125e+05
min	0.000000	0.000000	1986.000000	1988.000000	1995.000000	0.000000e+00
25%	6.000000	1.000000	1996.000000	1997.000000	2001.000000	1.027312e+06
50%	12.000000	5.000000	2002.000000	2006.000000	2007.000000	1.802568e+06
75%	19.000000	10.000000	2009.000000	2012.000000	2014.000000	2.443883e+06
max	25.000000	25.000000	2020.000000	2023.000000	2020.000000	3.999693e+06

In [9]:
```python
df.hist(figsize=(20,20))
plt.show()
```



In [10]:
```python
# Education Field of employees
df['Education'].value_counts()
```

Out[10]:
```
PG            6326
Doctorate     6285
Grad          6209
Under Grad    6180
Name: Education, dtype: int64
```

In [11]:
```python
# Gender of employees
df['Department'].value_counts()
```

Out[11]:
```
Marketing         2379
Analytics/BI      2096
Healthcare        2062
Others            2041
Sales             1991
HR                1988
Banking           1952
Education         1948
Engineering       1937
Top Management    1632
Accounts          1118
IT-Software       1078
Name: Department, dtype: int64
```

In [12]:
```python
# Employees in the database have several roles on-file
df['Role'].value_counts()
```

Out[12]:
```
Others                  2248
Bio statistician        1913
Analyst                 1892
Project Manager         1850
Team Lead               1833
Consultant              1780
Business Analyst        1711
Sales Execituve         1574
Sales Manager           1427
Senior Researcher       1236
Financial Analyst       1182
CEO                     1149
Scientist               1139
Head                    1108
Associate                767
Data scientist           363
Principal Analyst        275
Area Sales Manager       134
Senior Analyst           128
Researcher               123
Sr. Business Analyst     114
Professor                 33
Research Scientist        33
Lab Executuve             25
Name: Role, dtype: int64
```

In [13]:
```python
print(df.loc[df["Current_CTC"]==0])
```

```
       Total_Experience  Total_Experience_in_field_applied Department Role  \
0                     0                                  0        NaN  NaN
13                    0                                  0        NaN  NaN
140                   0                                  0        NaN  NaN
150                   0                                  0        NaN  NaN
176                   0                                  0        NaN  NaN
...                 ...                                ...        ...  ...
24850                 0                                  0        NaN  NaN
24903                 0                                  0        NaN  NaN
24926                 0                                  0        NaN  NaN
24959                 0                                  0        NaN  NaN
24970                 0                                  0        NaN  NaN

      Industry Organization Designation   Education Graduation_Specialization  \
0          NaN          NaN         NaN          PG                      Arts
13         NaN          NaN         NaN          PG               Engineering
140        NaN          NaN         NaN        Grad                    Others
150        NaN          NaN         NaN        Grad                    Botony
176        NaN          NaN         NaN          PG                    Others
...        ...          ...         ...         ...                       ...
24850      NaN          NaN         NaN  Under Grad                       NaN
24903      NaN          NaN         NaN        Grad                Psychology
24926      NaN          NaN         NaN        Grad                      Arts
24959      NaN          NaN         NaN          PG                 Chemistry
24970      NaN          NaN         NaN        Grad                Statistics

      University_Grad  ...  Curent_Location Preferred_location Current_CTC  \
0             Lucknow  ...         Guwahati               Pune           0
13             Nagpur  ...           Nagpur          Bangalore           0
140            Mumbai  ...          Lucknow            Lucknow           0
150              Pune  ...            Delhi          Bangalore           0
176           Lucknow  ...          Chennai               Pune           0
...               ...  ...              ...                ...         ...
24850             NaN  ...        Mangalore            Chennai           0
24903       Bangalore  ...          Chennai            Lucknow           0
24926           Surat  ...             Pune             Nagpur           0
24959       Bangalore  ...        Ahmedabad          Bangalore           0
24970       Bangalore  ...        Ahmedabad        Bhubaneswar           0

      Inhand_Offer Last_Appraisal_Rating No_Of_Companies_worked  \
0                N                   NaN                      0
13               N                   NaN                      0
140              N                   NaN                      0
150              N                   NaN                      0
176              N                   NaN                      0
...            ...                   ...                    ...
24850            N                   NaN                      0
24903            N                   NaN                      0
24926            N                   NaN                      0
24959            N                   NaN                      0
24970            N                   NaN                      0

      Number_of_Publications Certifications International_degree_any  \
0                          0              0                        0
13                         6              0                        0
140                        5              0                        0
150                        4              0                        0
176                        4              0                        0
...                      ...            ...                      ...
24850                      5              3                        0
24903                      7              0                        0
24926                      1              0                        0
24959                      7              1                        0
24970                      3              0                        0

      Expected_CTC
0           384551
13          639655
140         658618
150         546975
176         577554
...            ...
24850       358963
24903       573496
24926       314236
24959       886891
24970       597853

[908 rows x 27 columns]
```

In [14]:
```python
df.isnull().sum().sum()
```

Out[14]: 86853

In [15]:
```python
df["Passing_Year_Of_PHD"].replace(to_replace = np.nan, value = 0,inplace=True)
df["University_PHD"].replace(to_replace = np.nan, value = "Indian",inplace=True)
df["PHD_Specialization"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Passing_Year_Of_PG"].replace(to_replace = np.nan, value = 0,inplace=True)
df["PG_Specialization"].replace(to_replace = np.nan, value = "others",inplace=True)
df["University_PG"].replace(to_replace = np.nan, value = "Indian",inplace=True)
df["Graduation_Specialization"].replace(to_replace = np.nan, value = "others",inplace=True)
df["University_Grad"].replace(to_replace = np.nan, value = "Indian",inplace=True)
df["Passing_Year_Of_Graduation"].replace(to_replace = np.nan, value = 0,inplace=True)
df["Designation"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Department"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Role"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Industry"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Organization"].replace(to_replace = np.nan, value = "others",inplace=True)
df["Last_Appraisal_Rating"].replace(to_replace = np.nan, value = "others",inplace=True)
```

In [16]:
```python
df.isnull().sum().sort_values(ascending = False)
```

Out[16]:
```
Total_Experience                     0
PHD_Specialization                   0
International_degree_any             0
Certifications                       0
Number_of_Publications               0
No_Of_Companies_worked               0
Last_Appraisal_Rating                0
Inhand_Offer                         0
Current_CTC                          0
Preferred_location                   0
Curent_Location                      0
Passing_Year_Of_PHD                  0
University_PHD                       0
Passing_Year_Of_PG                   0
Total_Experience_in_field_applied    0
University_PG                        0
PG_Specialization                    0
Passing_Year_Of_Graduation           0
University_Grad                      0
Graduation_Specialization            0
Education                            0
Designation                          0
Organization                         0
Industry                             0
Role                                 0
Department                           0
Expected_CTC                         0
dtype: int64
```

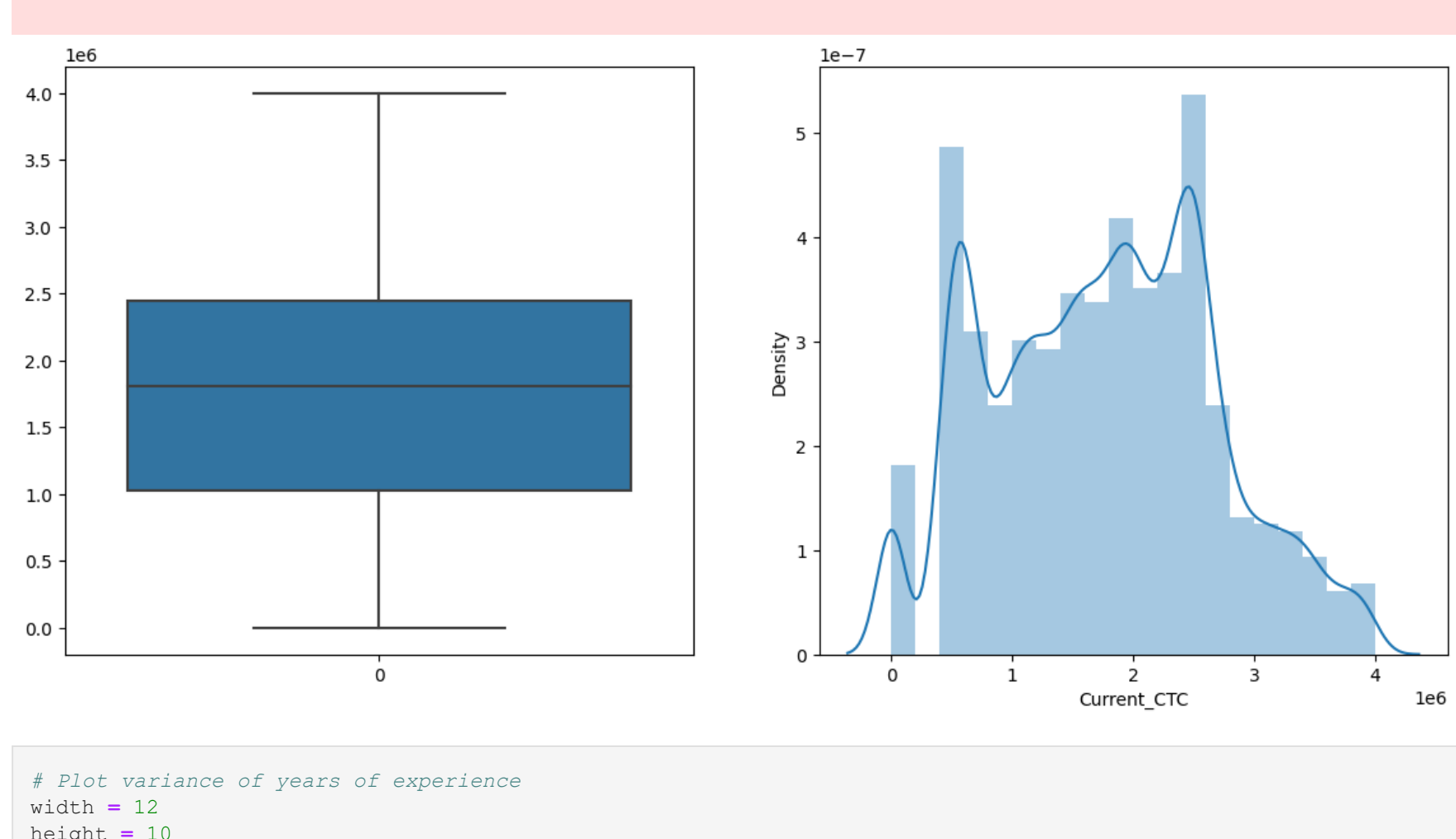
In [17]:
```python
#distribution  of salary
plt.figure(figsize = (14, 6))
plt.subplot(1, 2, 1)
sns.boxplot(df.Current_CTC )
plt.subplot(1, 2, 2)
sns.distplot(df.Current_CTC , bins = 20)
plt.show()
```

```
C:\Users\adity\anaconda3\envs\AntonioEnv\lib\site-packages\ipykernel_launcher.py:6: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```



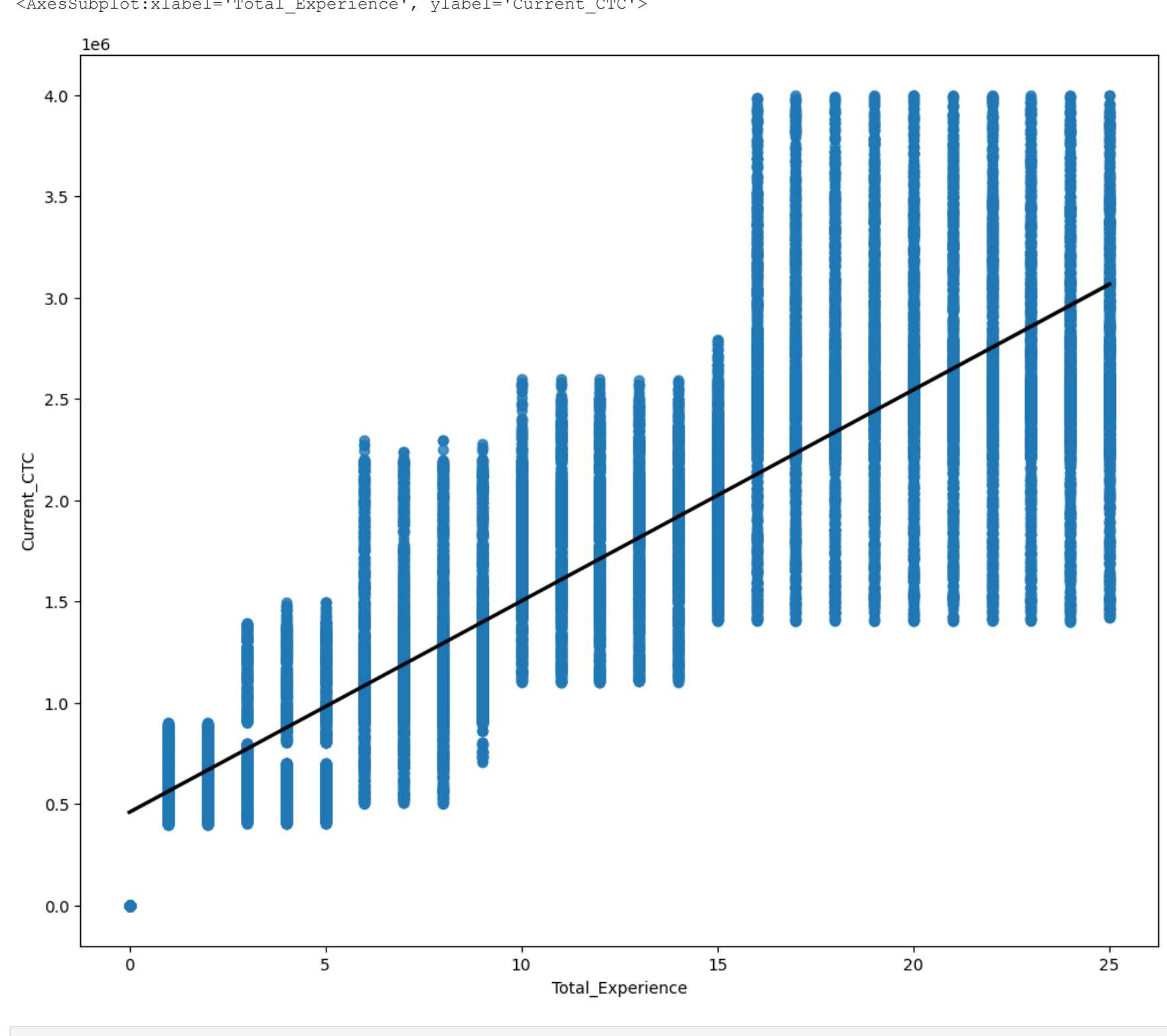
In [18]:
```python
# Plot variance of years of experience
width = 12
height = 10
plt.figure(figsize=(width, height))
sns.residplot(df['Total_Experience_in_field_applied'], df['Current_CTC'])
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3780\3814394127.py in <module>
      3 height = 10
      4 plt.figure(figsize=(width, height))
----> 5 sns.residplot(df['Total_Experience_in_field_applied'], df['Current_CTC'])
      6 plt.show()

TypeError: residplot() takes from 0 to 1 positional arguments but 2 were given
<Figure size 1200x1000 with 0 Axes>
```

In [19]:
```python
# Plot regression line on years of experience
width = 12
height = 10
plt.figure(figsize=(width, height))
sns.regplot(x='Total_Experience', y='Current_CTC', data=df, line_kws={'color':'black'})
```

Out[19]: <AxesSubplot:xlabel='Total_Experience', ylabel='Current_CTC'>



In [20]:
```python
# Plot variance of years of experience
width = 12
height = 10
plt.figure(figsize=(width, height))
sns.residplot(df['Total_Experience'], df['Current_CTC'])
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3780\1715822780.py in <module>
      3 height = 10
      4 plt.figure(figsize=(width, height))
----> 5 sns.residplot(df['Total_Experience'], df['Current_CTC'])
      6 plt.show()

TypeError: residplot() takes from 0 to 1 positional arguments but 2 were given
<Figure size 1200x1000 with 0 Axes>
```

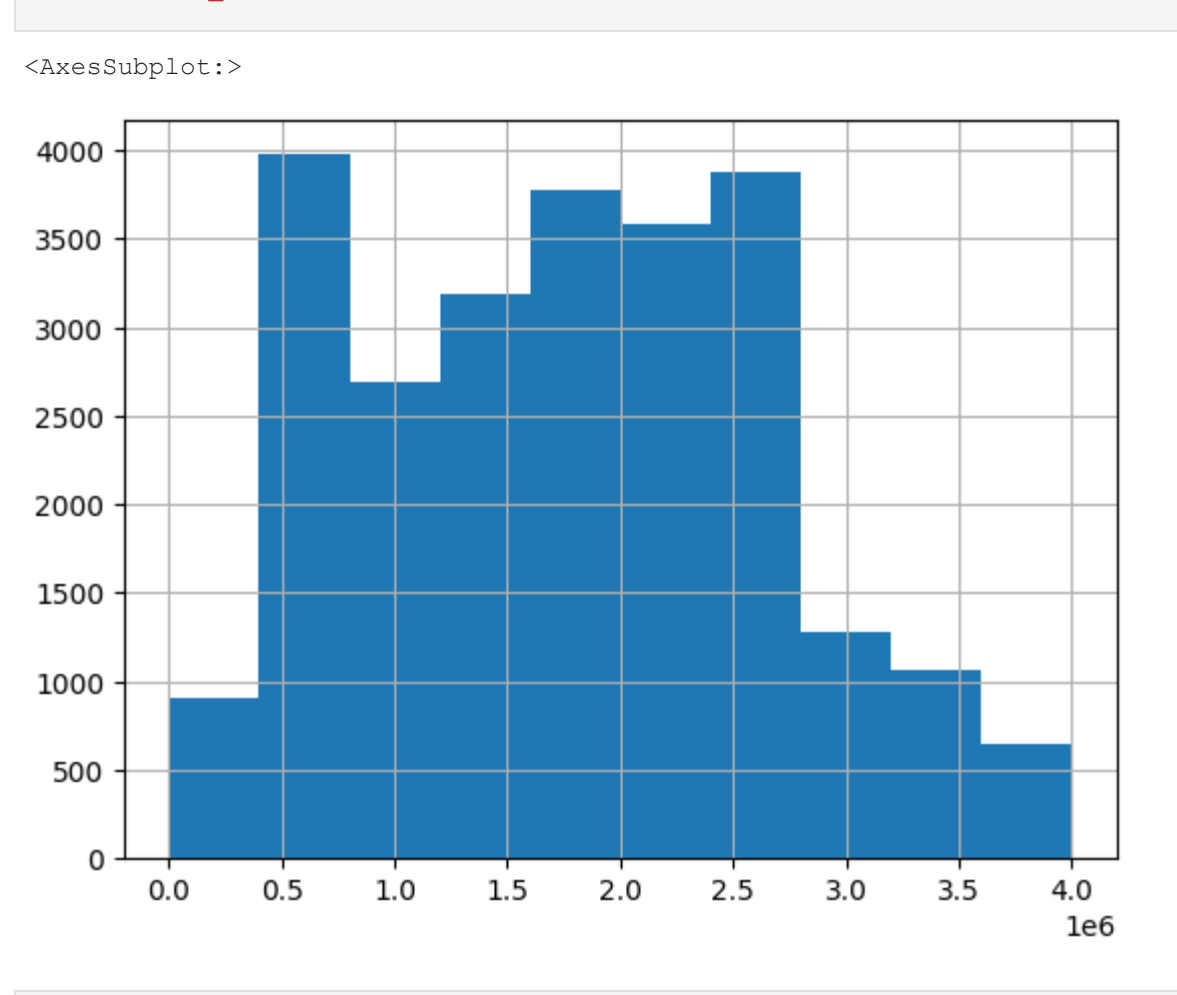
In [ ]:
```python
# Plot boxplots for companyId
width = 16
height = 8
var = 'Organization'
data = pd.concat([df['Current_CTC'], df[var]], axis=1)
f, ax = plt.subplots(figsize=(width, height))
fig = sns.boxplot(x=var, y='Current_CTC', data=df)
plt.xticks(rotation=90)
```

In [ ]:
```python
f, ax = plt.subplots(figsize=(12, 9));
sns.barplot('Current_CTC','Role',data=df)
```

In [ ]:
```python
a = df["Current_CTC"][:10].plot.barh(figsize=(16,10))
```

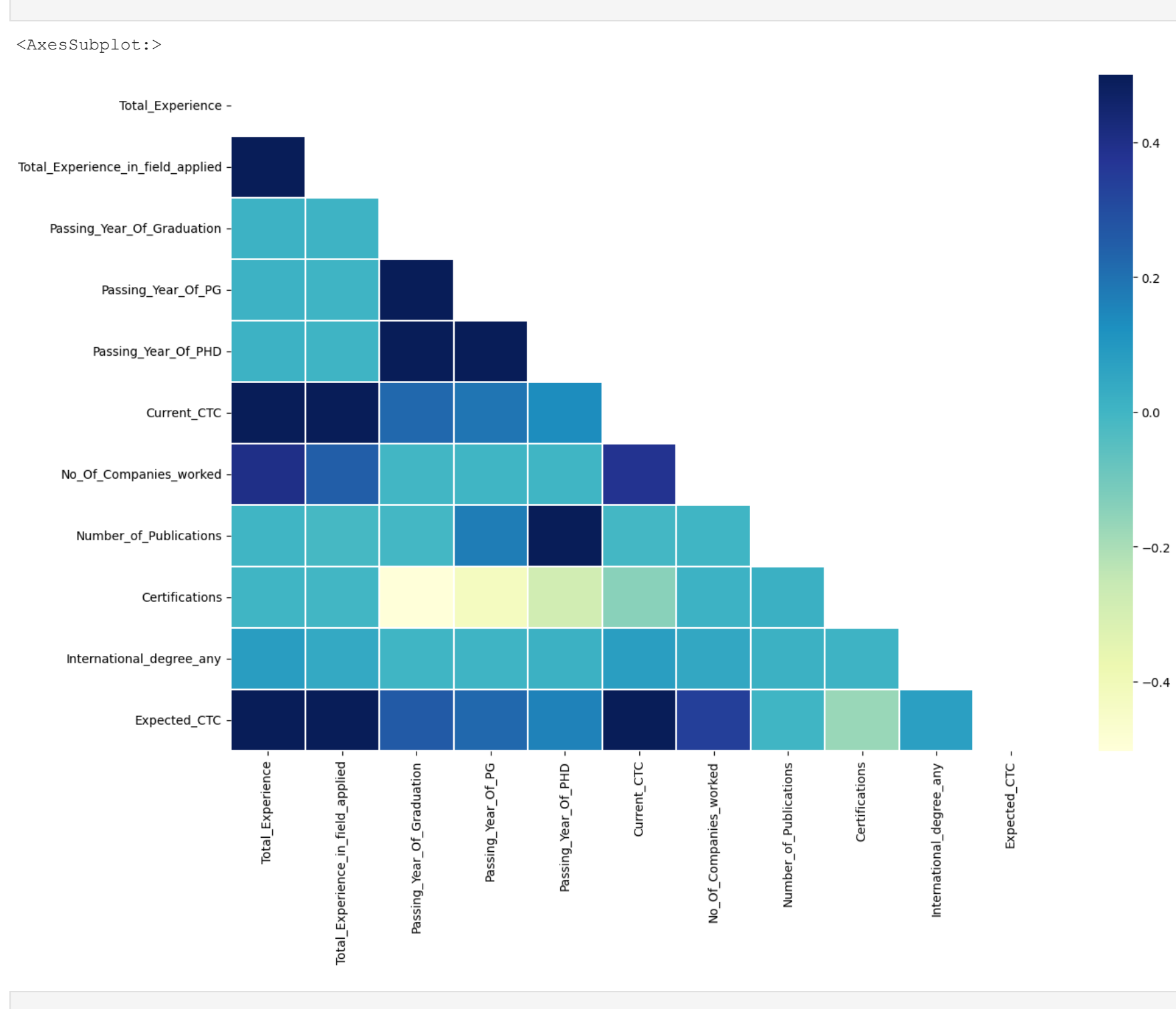
In [21]:
```python
df["Current_CTC"].hist()
```

Out[21]: <AxesSubplot:>



In [22]:
```python
#Calculate correlations
corr = df.corr()
mask = np.zeros_like(corr)
mask[np.triu_indices_from(mask)] = True
# Heatmap
plt.figure(figsize=(15, 10))
sns.heatmap(corr,
            vmax=.5,
            mask=mask,
            # annot=True, fmt='.2f',
            linewidths=.2, cmap="YlGnBu")
```

Out[22]: <AxesSubplot:>

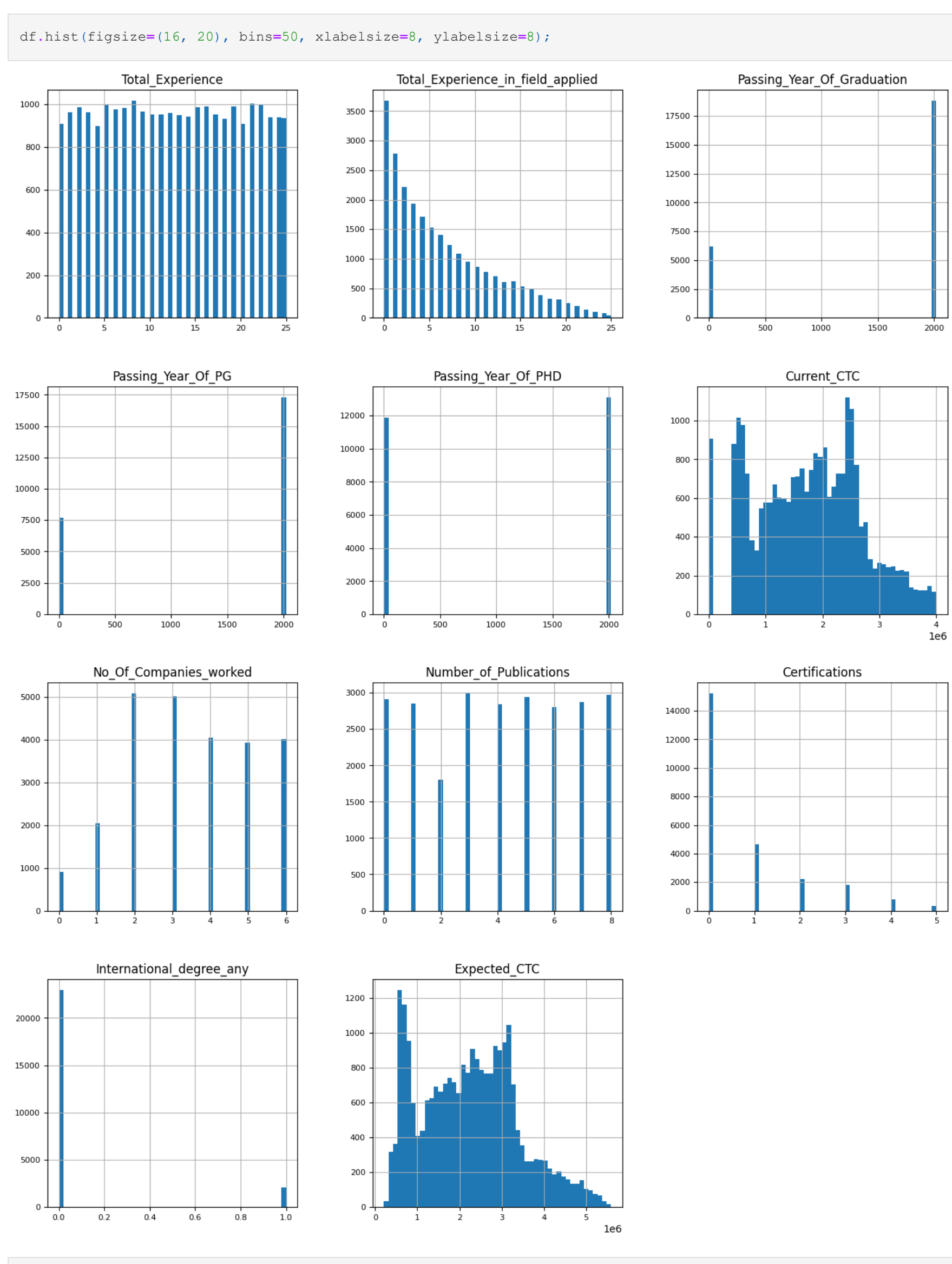


In [23]:
```python
df_num = df.select_dtypes(include = ['float64', 'int64'])
df_num.head()
```

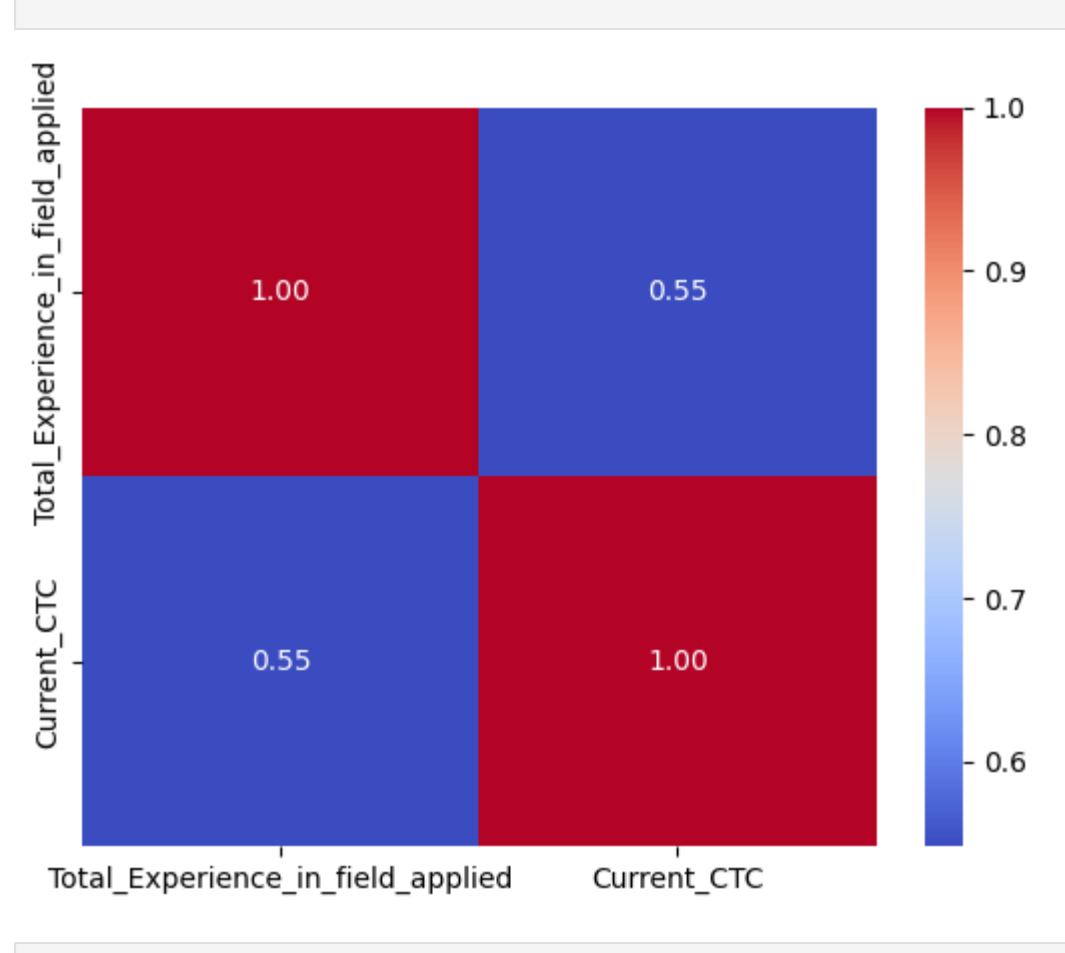
Out[23]:

	Total_Experience	Total_Experience_in_field_applied	Passing_Year_Of_Graduation	Passing_Year_Of_PG	Passing_Year_Of_PHD	Current_CTC	No_Of
0	0	0	2020.0	0.0	0.0	0	
1	23	14	1988.0	1990.0	1997.0	2702664	
2	21	12	1990.0	1992.0	1999.0	2236661	
3	15	8	1997.0	1999.0	2005.0	2100510	
4	10	5	2004.0	2006.0	2010.0	1931644	

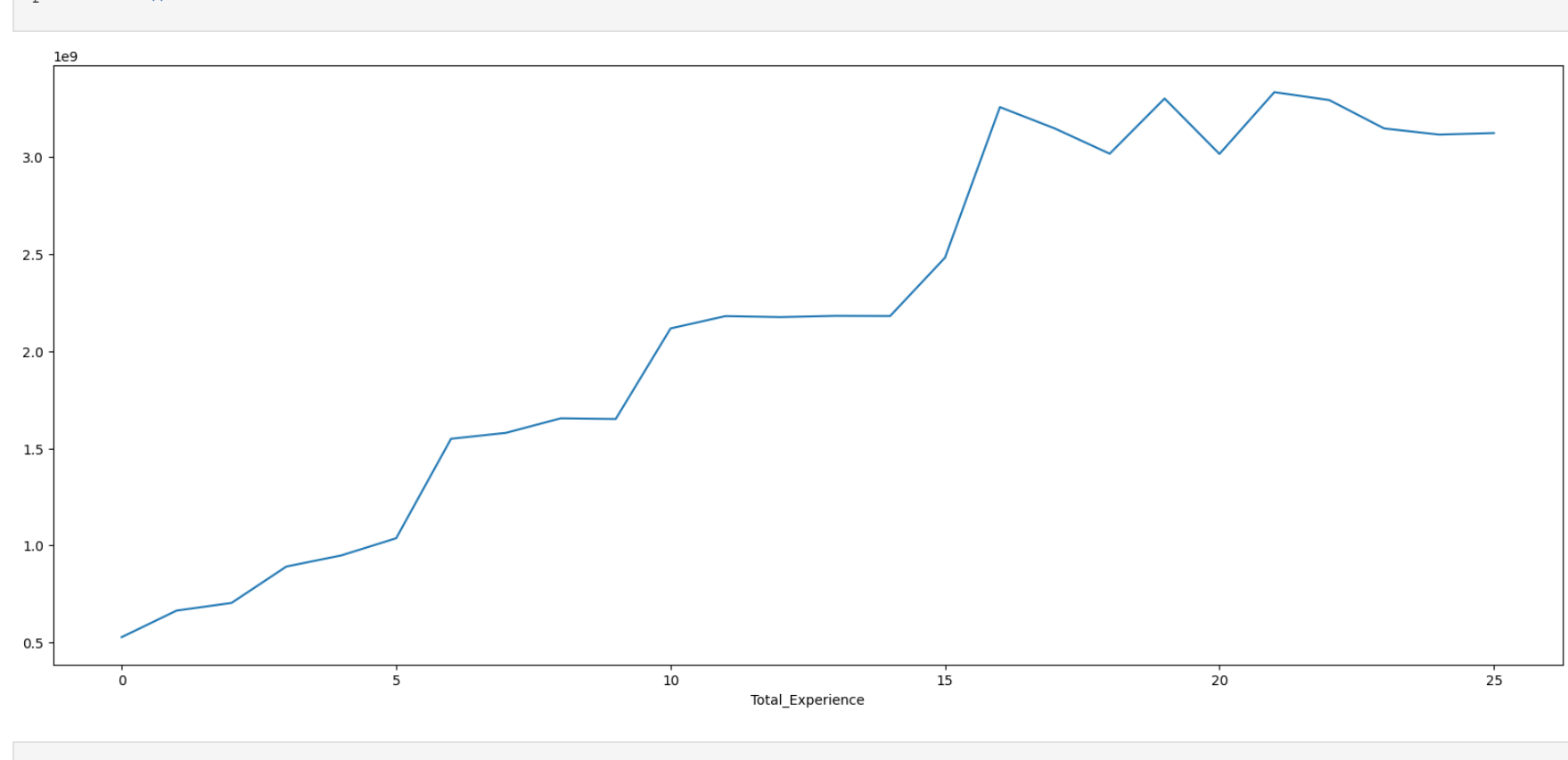
In [24]:
```python
df.hist(figsize=(16, 20), bins=50, xlabelsize=8, ylabelsize=8);
```



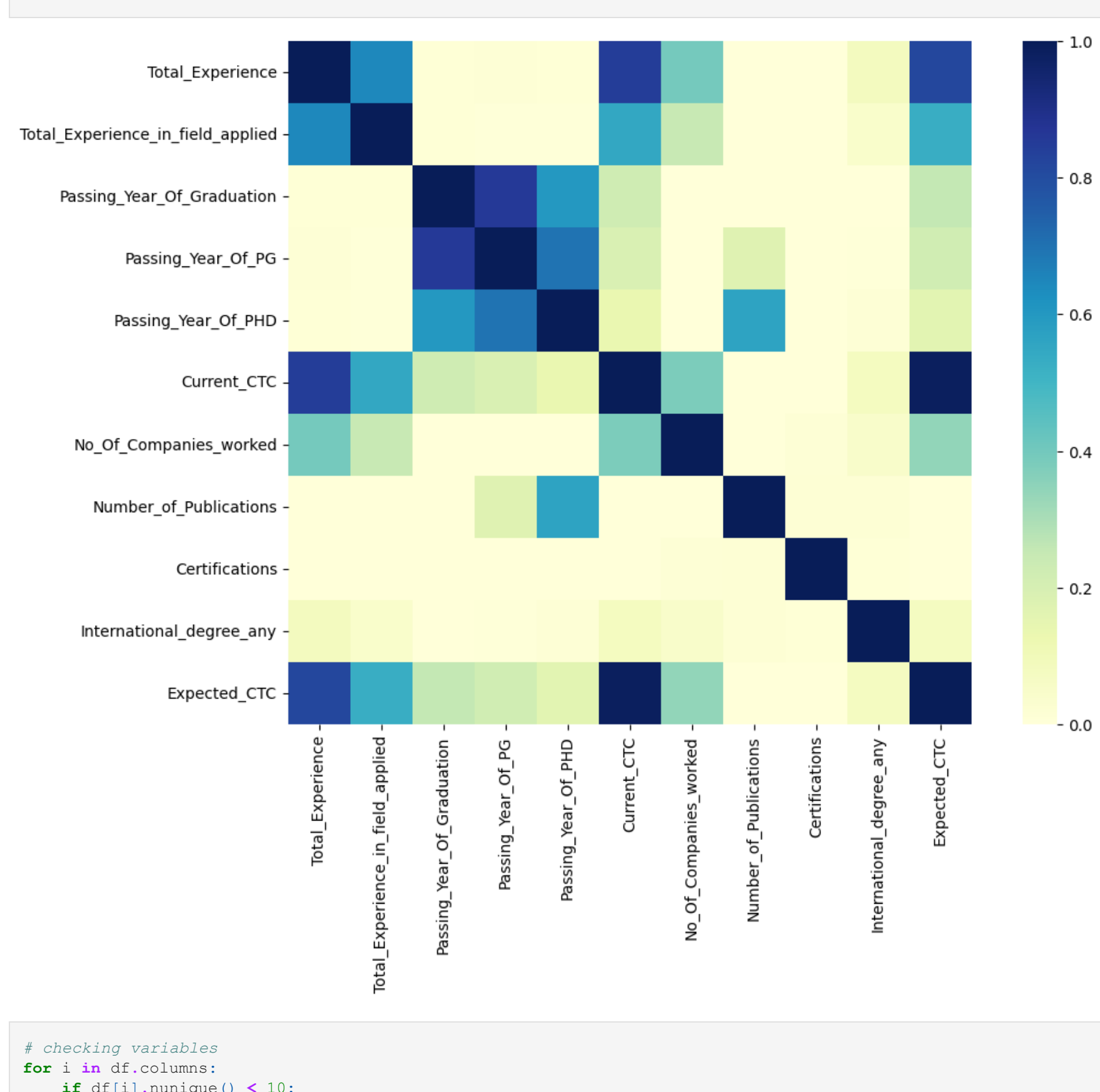
In [25]:
```python
#Correlation Matrix
g=sns.heatmap(df[["Total_Experience_in_field_applied","Current_CTC","Graduation_Specialization"]].corr(),annot=
```



In [26]:
```python
#Time series view
plt.figure(figsize=(20,8))
df.groupby(['Total_Experience'])['Expected_CTC'].sum().plot(kind='line')
plt.show()
```



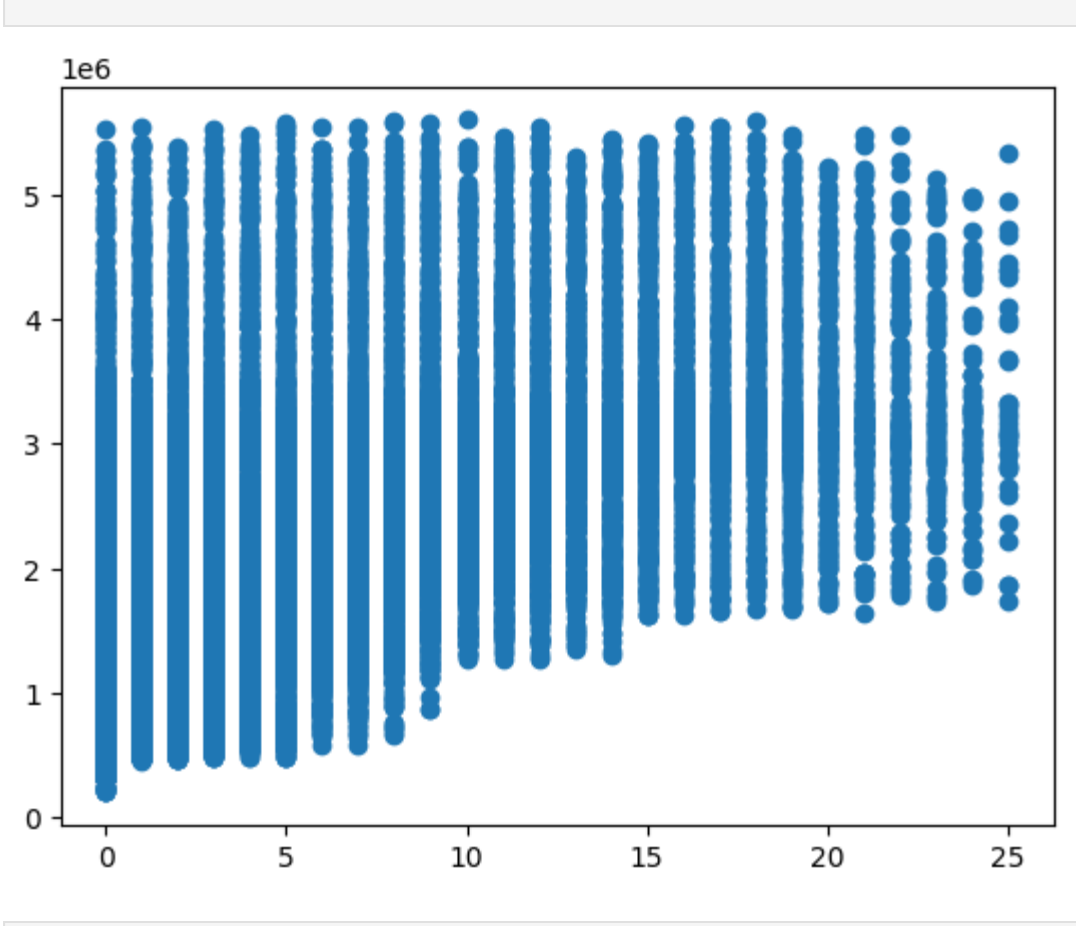
In [27]:
```python
Corr = df.corr()
plt.figure(figsize=(12,8))
sns.heatmap(data=corr,vmin=0, vmax=1, cmap="YlGnBu",  square=True)
plt.show()
```



In [28]:
```python
# checking variables
for i in df.columns:
    if df[i].nunique() < 10:
        print(f'The column "{i}" is __{df[i].dtype}__ \nhas __{df[i].nunique()}__ unique values: \n{df[i].value
        print(10*'==')
    else:
        print(f'The column "{i}" is __{df[i].dtype}__ \nhas __{df[i].nunique()}__ unique values')
        print(10*'==')
```

```
The column "Total_Experience" is __int64__
has __26__ unique values
====================
The column "Total_Experience_in_field_applied" is __int64__
has __26__ unique values
====================
The column "Department" is __object__
has __13__ unique values
====================
The column "Role" is __object__
has __25__ unique values
====================
The column "Industry" is __object__
has __12__ unique values
====================
The column "Organization" is __object__
has __17__ unique values
====================
The column "Designation" is __object__
has __19__ unique values
====================
The column "Education" is __object__
has __4__ unique values:
PG            6326
Doctorate     6285
Grad          6209
Under Grad    6180
Name: Education, dtype: int64
====================
The column "Graduation_Specialization" is __object__
has __12__ unique values
====================
The column "University_Grad" is __object__
has __14__ unique values
====================
The column "Passing_Year_Of_Graduation" is __float64__
has __36__ unique values
====================
The column "PG_Specialization" is __object__
has __12__ unique values
====================
The column "University_PG" is __object__
has __14__ unique values
====================
The column "Passing_Year_Of_PG" is __float64__
has __37__ unique values
====================
The column "PHD_Specialization" is __object__
has __12__ unique values
====================
The column "University_PHD" is __object__
has __14__ unique values
====================
The column "Passing_Year_Of_PHD" is __float64__
has __27__ unique values
====================
The column "Curent_Location" is __object__
has __15__ unique values
====================
The column "Preferred_location" is __object__
has __15__ unique values
====================
The column "Current_CTC" is __int64__
has __23992__ unique values
====================
The column "Inhand_Offer" is __object__
has __2__ unique values:
N    17418
Y     7582
Name: Inhand_Offer, dtype: int64
====================
The column "Last_Appraisal_Rating" is __object__
has __6__ unique values:
B                 5501
D                 4917
C                 4812
A                 4671
Key_Performer     4191
others             908
Name: Last_Appraisal_Rating, dtype: int64
====================
The column "No_Of_Companies_worked" is __int64__
has __7__ unique values:
2    5076
3    5006
4    4037
6    4012
5    3925
1    2036
0     908
Name: No_Of_Companies_worked, dtype: int64
====================
The column "Number_of_Publications" is __int64__
has __9__ unique values:
3    2990
8    2970
5    2941
0    2912
7    2873
1    2856
4    2846
6    2804
2    1808
Name: Number_of_Publications, dtype: int64
====================
The column "Certifications" is __int64__
has __6__ unique values:
0    15215
1     4644
2     2198
3     1818
4      777
5      348
Name: Certifications, dtype: int64
====================
The column "International_degree_any" is __int64__
has __2__ unique values:
0    22957
1     2043
Name: International_degree_any, dtype: int64
====================
The column "Expected_CTC" is __int64__
has __24913__ unique values
====================
```

In [29]:
```
plt.scatter(df.Total_Experience_in_field_applied,df.Expected_CTC)
plt.show()
```



In [30]:
```
from sklearn.preprocessing import LabelEncoder
```

In [31]:
```
from sklearn.model_selection import train_test_split
```

In [32]:
```
cols = ['Department', 'Role', 'Industry', 'Organization', 'Designation',
       'Education', 'Graduation_Specialization', 'University_Grad',
       'Passing_Year_Of_Graduation', 'PG_Specialization', 'University_PG',
       'Passing_Year_Of_PG', 'PHD_Specialization', 'University_PHD',
       'Passing_Year_Of_PHD', 'Curent_Location', 'Preferred_location', 'Inhand_Offer', 'Last_Appraisal_Rating',
       'No_Of_Companies_worked', 'Number_of_Publications', 'Certifications',
       'International_degree_any']


df[cols] = df[cols].apply(LabelEncoder().fit_transform)


df.head()
```

Out[32]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduation_Specializati
0	0	0	12	24	11	16	18	2	
1	23	14	5	6	0	7	5	0	
2	21	12	11	6	10	9	18	0	
3	15	8	2	8	2	5	5	0	
4	10	5	10	14	6	4	8	1	

5 rows × 27 columns

In [33]:
```
categorical_features=[ x for x in df.columns if df[x].dtype == 'O']
categorical_features
```

Out[33]: []

In [34]:
```
#finding count of different elements in the column

for x in categorical_features:
    a = df[x].value_counts()
    print(a)
```

In [35]:
```
numerical_features = [x for x in df.columns if x not in categorical_features]
numerical_features
```

Out[35]:
```
['Total_Experience',
 'Total_Experience_in_field_applied',
 'Department',
 'Role',
 'Industry',
 'Organization',
 'Designation',
 'Education',
 'Graduation_Specialization',
 'University_Grad',
 'Passing_Year_Of_Graduation',
 'PG_Specialization',
 'University_PG',
 'Passing_Year_Of_PG',
 'PHD_Specialization',
 'University_PHD',
 'Passing_Year_Of_PHD',
 'Curent_Location',
 'Preferred_location',
 'Current_CTC',
 'Inhand_Offer',
 'Last_Appraisal_Rating',
 'No_Of_Companies_worked',
 'Number_of_Publications',
 'Certifications',
 'International_degree_any',
 'Expected_CTC']
```

In [36]:
```
from pandas import factorize
```

In [37]:
```
# corelation value
labels, categories = factorize(df["Expected_CTC"])
df["Designation"] = labels
df["Designation"].corr(df["Expected_CTC"])
```

Out[37]: -0.004798571302144544

In [38]:
```
labels, categories = factorize(df["Designation"])
df["Certifications"] = labels
df["Number_of_Publications"].corr(df["Designation"])
```

Out[38]: 0.0030449293046329123

In [39]:
```
df.groupby(['Total_Experience','Department'])['Expected_CTC'].sum().sort_values(ascending=False)
```

Out[39]:
```
Total_Experience  Department
0                 12            528464569
19                9             376828188
17                11            342869598
22                11            327655625
24                1             325005619
                                   ...
5                 7              36715254
1                 0              32385472
                  7              30765922
3                 7              29673169
2                 7              26771716
Name: Expected_CTC, Length: 321, dtype: int64
```

In [40]:
```
#skewness and kurtosis
print("Skewness: %f" % df['Expected_CTC'].skew())
print("Kurtosis: %f" % df['Expected_CTC'].kurt())
```

```
Skewness: 0.331972
Kurtosis: -0.500660
```

In [41]:
```
print("Skewness: %f" % df['Industry'].skew())
print("Kurtosis: %f" % df['Industry'].kurt())
```

```
Skewness: 0.025820
Kurtosis: -1.194177
```

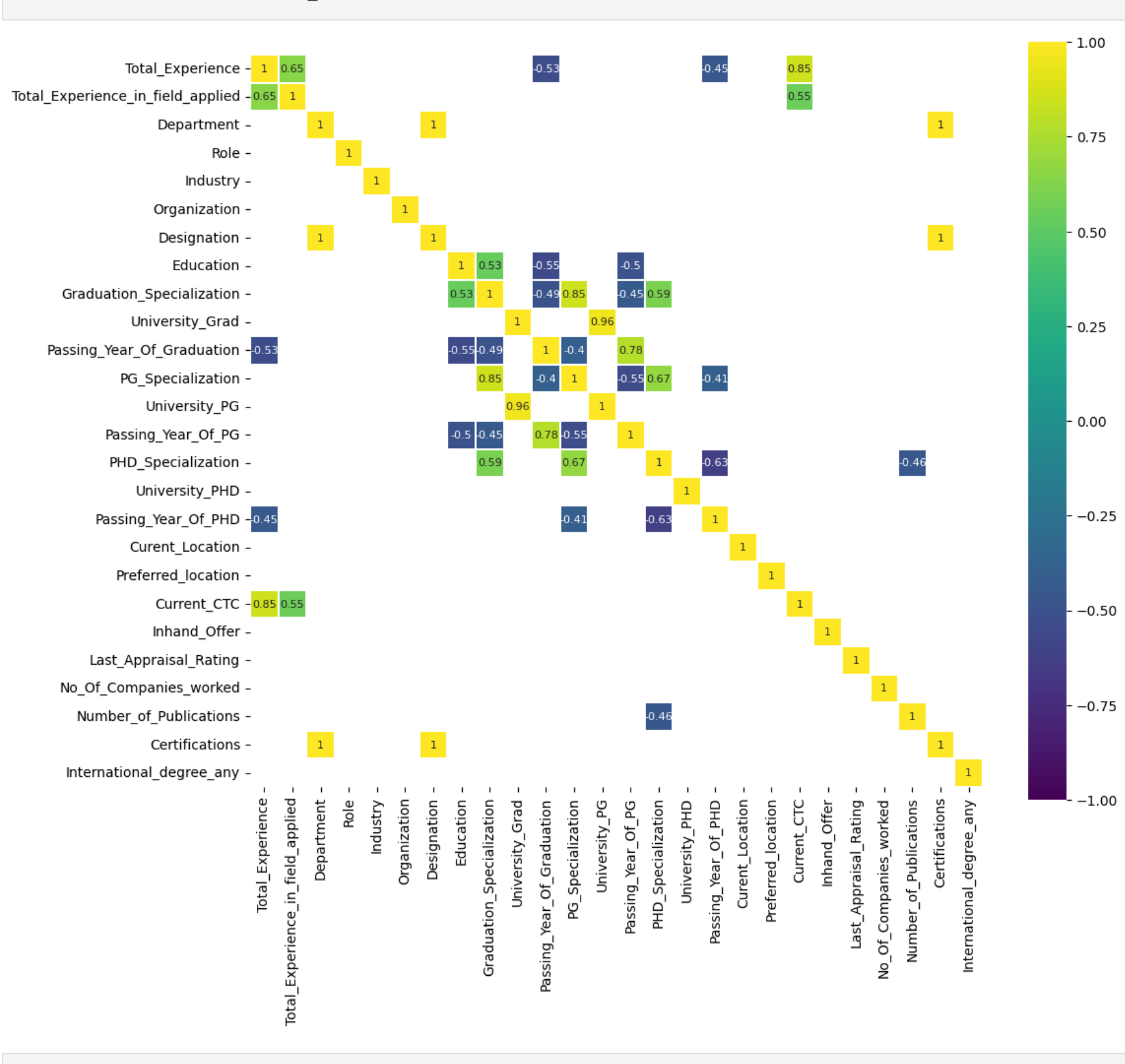
In [42]:
```
# corelation value
labels, categories = factorize(df["Expected_CTC"])
df["Department"] = labels
df["Expected_CTC"].corr(df["Expected_CTC"])
```

Out[42]: 1.0

In [43]:
```
#feature to Feature relation

corr = df.drop('Expected_CTC', axis=1).corr() # We already examined total revenue correlations
plt.figure(figsize=(12, 10))

sns.heatmap(corr[(corr >= 0.5) | (corr <= -0.4)],
            cmap='viridis', vmax=1.0, vmin=-1.0, linewidths=0.1,
            annot=True, annot_kws={"size": 8}, square=True);
```



In [44]:
```
from sklearn.preprocessing import StandardScaler,LabelEncoder,MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
from sklearn.model_selection import GridSearchCV

from sklearn.ensemble import RandomForestRegressor
```

In [45]:
```
str_col=['Total_Experience','No_Of_Companies_worked','Expected_CTC']
le=LabelEncoder()
for col in str_col:
    df[col]=le.fit_transform(df[col].values)
```

In [46]:
```
sc=MinMaxScaler()
num_col=list(df.select_dtypes(exclude = ['object']).drop(columns=['Expected_CTC',]).columns)
df_scaled=sc.fit_transform(df[num_col].values)
```

In [47]:
```
df_scaled=pd.DataFrame(df_scaled,columns=num_col)
df_scaled.head()
```

Out[47]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduation_Special
0	0.00	0.00	0.000000	1.000000	1.000000	1.0000	0.000000	0.666667	0.
1	0.92	0.56	0.000040	0.250000	0.000000	0.4375	0.000040	0.000000	0.
2	0.84	0.48	0.000080	0.250000	0.909091	0.5625	0.000080	0.000000	0.
3	0.60	0.32	0.000120	0.333333	0.181818	0.3125	0.000120	0.000000	0.
4	0.40	0.20	0.000161	0.583333	0.545455	0.2500	0.000161	0.333333	0.

5 rows × 26 columns

In [48]:
```
df[num_col]
```

Out[48]:

	Total_Experience	Total_Experience_in_field_applied	Department	Role	Industry	Organization	Designation	Education	Graduation_Specia
0	0	0	0	24	11	16	0	2	
1	23	14	1	6	0	7	1	0	
2	21	12	2	6	10	9	2	0	
3	15	8	3	8	2	5	3	0	
4	10	5	4	14	6	4	4	1	
...	...	...	...	...	...	...	...	...	
24995	18	13	24908	14	1	8	24908	2	
24996	12	8	24909	11	0	1	24909	3	
24997	22	8	24910	9	6	3	24910	3	
24998	25	8	24911	5	3	3	24911	2	
24999	8	0	24912	6	1	15	24912	1	

25000 rows × 26 columns

In [49]:
```
# Adding test and traing flag so that we can split train/test data
df['tst']=0
df['tst']=1
```

In [50]:
```
train=df[df.tst==0]
test=df[df.tst==1]
```

In [51]:
```
train=train.drop(columns=['tst','Department'],axis=1)
```

In [52]:
```
test=test.drop(columns=['tst','Expected_CTC','Department'],axis=1)
```

In [53]:
```
X=train.columns.drop('Expected_CTC')
y=train.Expected_CTC
X
```

Out[53]:
```
Index(['Total_Experience', 'Total_Experience_in_field_applied', 'Role',
       'Industry', 'Organization', 'Designation', 'Education',
       'Graduation_Specialization', 'University_Grad',
       'Passing_Year_Of_Graduation', 'PG_Specialization', 'University_PG',
       'Passing_Year_Of_PG', 'PHD_Specialization', 'University_PHD',
       'Passing_Year_Of_PHD', 'Curent_Location', 'Preferred_location',
       'Current_CTC', 'Inhand_Offer', 'Last_Appraisal_Rating',
       'No_Of_Companies_worked', 'Number_of_Publications', 'Certifications',
       'International_degree_any'],
      dtype='object')
```

In [54]:
```
train[X].var().sort_values(ascending=False)
```

Out[54]:
```
Total_Experience                     NaN
Total_Experience_in_field_applied    NaN
Role                                 NaN
Industry                             NaN
Organization                         NaN
Designation                          NaN
Education                            NaN
Graduation_Specialization            NaN
University_Grad                      NaN
Passing_Year_Of_Graduation           NaN
PG_Specialization                    NaN
University_PG                        NaN
Passing_Year_Of_PG                   NaN
PHD_Specialization                   NaN
University_PHD                       NaN
Passing_Year_Of_PHD                  NaN
Curent_Location                      NaN
Preferred_location                   NaN
Current_CTC                          NaN
Inhand_Offer                         NaN
Last_Appraisal_Rating                NaN
No_Of_Companies_worked               NaN
Number_of_Publications               NaN
Certifications                       NaN
International_degree_any             NaN
dtype: float64
```

In [55]:
```
X = df[['Education','Total_Experience','Organization']]
Y = df['Expected_CTC']
```

In [56]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.3,random_state=1)
```

In [57]:
```
X_train.shape,X_test.shape,Y_train.shape,Y_test.shape
```

Out[57]: ((17500, 3), (7500, 3), (17500,), (7500,))

In [58]:
```
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

In [59]:
```
from sklearn.linear_model import LinearRegression
```

In [60]:
```
lr = LinearRegression()
```

In [61]:
```
lr.fit(X_train_scaled,Y_train)
```

Out[61]: LinearRegression()

In [62]:
```
# View coefficients of linear regression object
print(lr.intercept_)
print(lr.coef_)
```

```
12447.9972
[-1923.57519545  6001.48671816   -30.83535964]
```

In [63]:
```
#Slopes/Coefficients
m = lr.coef_
print("slopes  : ",m[0].round(2))
print("slopes  : ",m[1].round(2))

#Intercept
c = lr.intercept_
print("Intercept is : ",c.round(2))
```

```
slopes  :  -1923.58
slopes  :  6001.49
Intercept is :  12448.0
```

In [64]:
```
Y_pred = lr.predict(X_test_scaled)
```

In [65]:
```
Y_pred
```

Out[65]:
```
array([ 9732.42428075, 12758.58600539, 18570.76615236, ...,
       14511.21623641, 15900.58265938,  6540.12770313])
```

In [66]:
```
#compre actual vs predicted o/p
dict={'Actual Output':Y_test,'Predicted output':Y_pred}
#convert dictionary into dataframe
df1=pd.DataFrame(dict)
df1
```

Out[66]:

	Actual Output	Predicted output
21492	5693	9732.424281
9488	9670	12758.586005
16933	16669	18570.766152
12604	2244	6507.951334
8222	1345	6609.733448
...	...	...
13410	6713	7419.069322
13158	21196	11931.126577
3552	13259	14511.216236
23203	21492	15900.582659
2410	3029	6540.127703

7500 rows × 2 columns

In [67]:
```
#Mean Squared Error 1/n*sum(Y-Y-pred)**2
from sklearn.metrics import mean_squared_error
print("MSE: ",mean_squared_error(Y_test,Y_pred))
```

```
MSE:  11914771.154343156
```

In [68]:
```
#Root mean squred error
mse=mean_squared_error(Y_test,Y_pred)
rmse=np.sqrt(mse)
print("Root Mean Sqaured Error : ",rmse)
```

```
Root Mean Sqaured Error :  3451.7779700240217
```

In [69]:
```
#R2--Score
#Check model is perfect or not
from sklearn.metrics import r2_score
print("R2-score : ",r2_score(Y_test,Y_pred))
```

```
R2-score :  0.7712128103467416
```

In [70]:
```
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
```

In [71]:
```
print("MAE",mean_absolute_error(Y_test,Y_pred))
print("MSE",mean_squared_error(Y_test,Y_pred))
print("R2 score",r2_score(Y_test,Y_pred))
```

```
MAE 2636.8755612090786
MSE 11914771.154343156
R2 score 0.7712128103467416
```

In [72]:
```
#Find Error or Residue
residual=Y_test-Y_pred
print(residual.head())
```

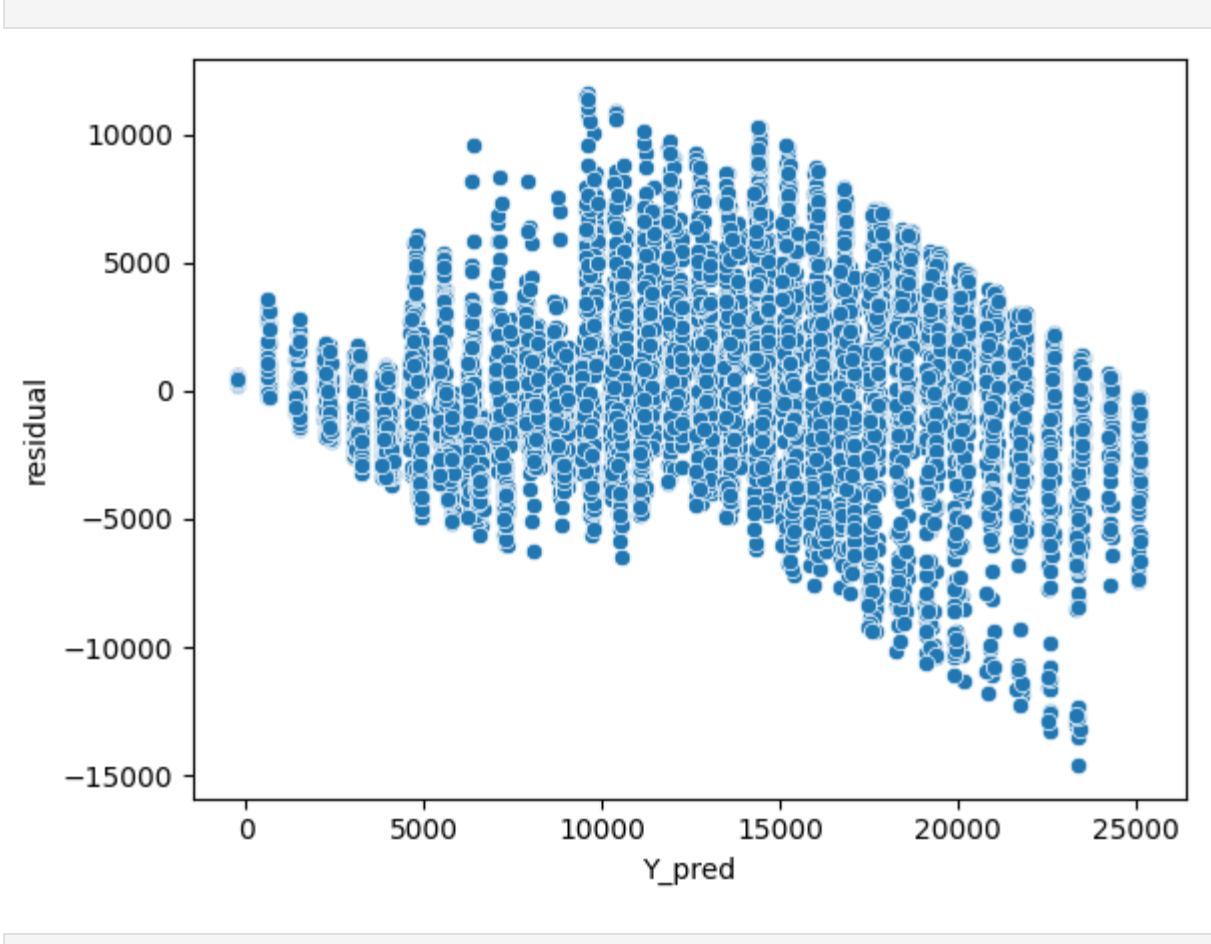
```
21492   -4039.424281
9488    -3088.586005
16933   -1901.766152
12604   -4263.951334
8222    -5264.733448
Name: Expected_CTC, dtype: float64
```

In [73]:
```
#Create a Dictionary:
dic={'Y_pred' :Y_pred,'residual' : residual}
df2=pd.DataFrame(dic)
df2.head()
```

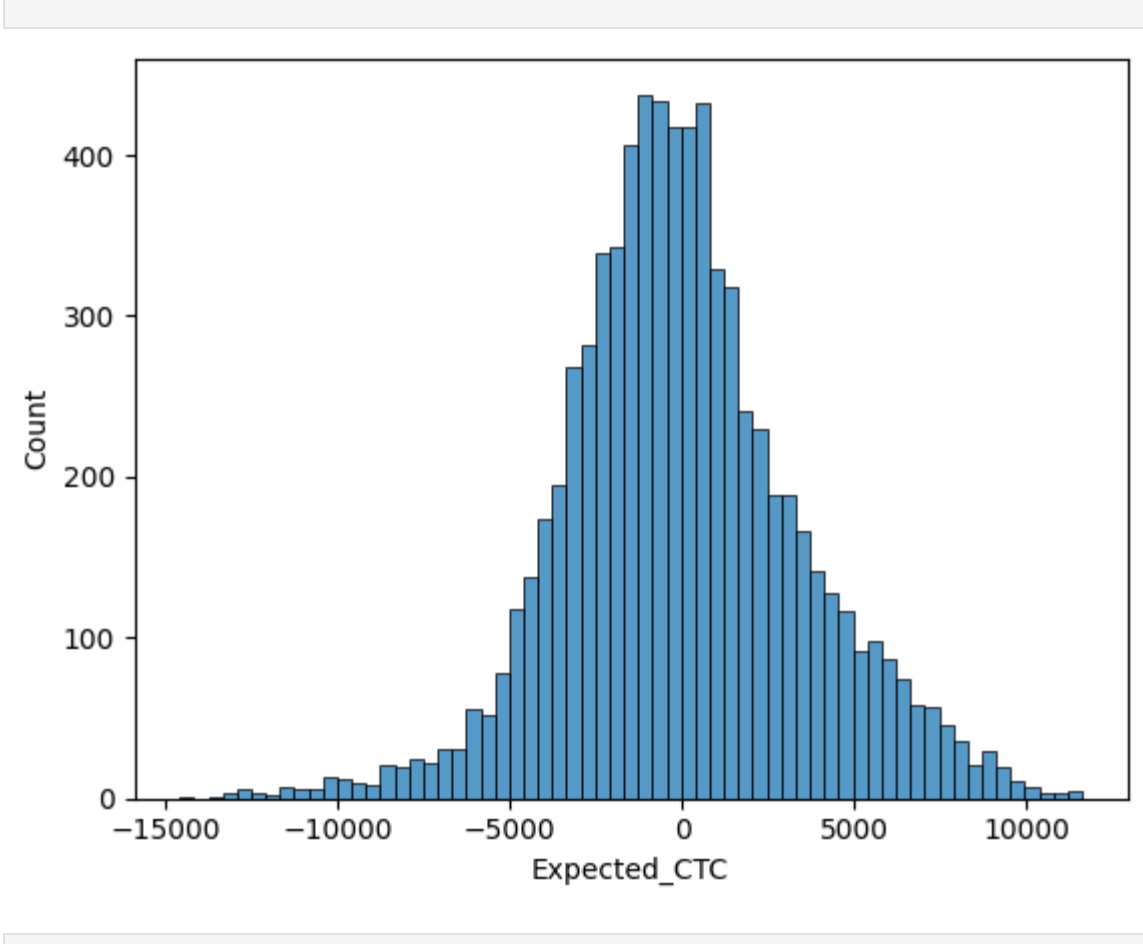
Out[73]:

	Y_pred	residual
21492	9732.424281	-4039.424281
9488	12758.586005	-3088.586005
16933	18570.766152	-1901.766152
12604	6507.951334	-4263.951334
8222	6609.733448	-5264.733448

In [74]:
```
#Visualise scatterplot between Y_pred & residual means
#here input x=Y_pred & output y=residual because residual depends on Y_pred value.
sns.scatterplot(data=df2,x='Y_pred',y='residual')
plt.show()
```



In [75]:
```
#create Histogram(Frequency graph):
#here ngative skewness
sns.histplot(residual)
plt.show()
```



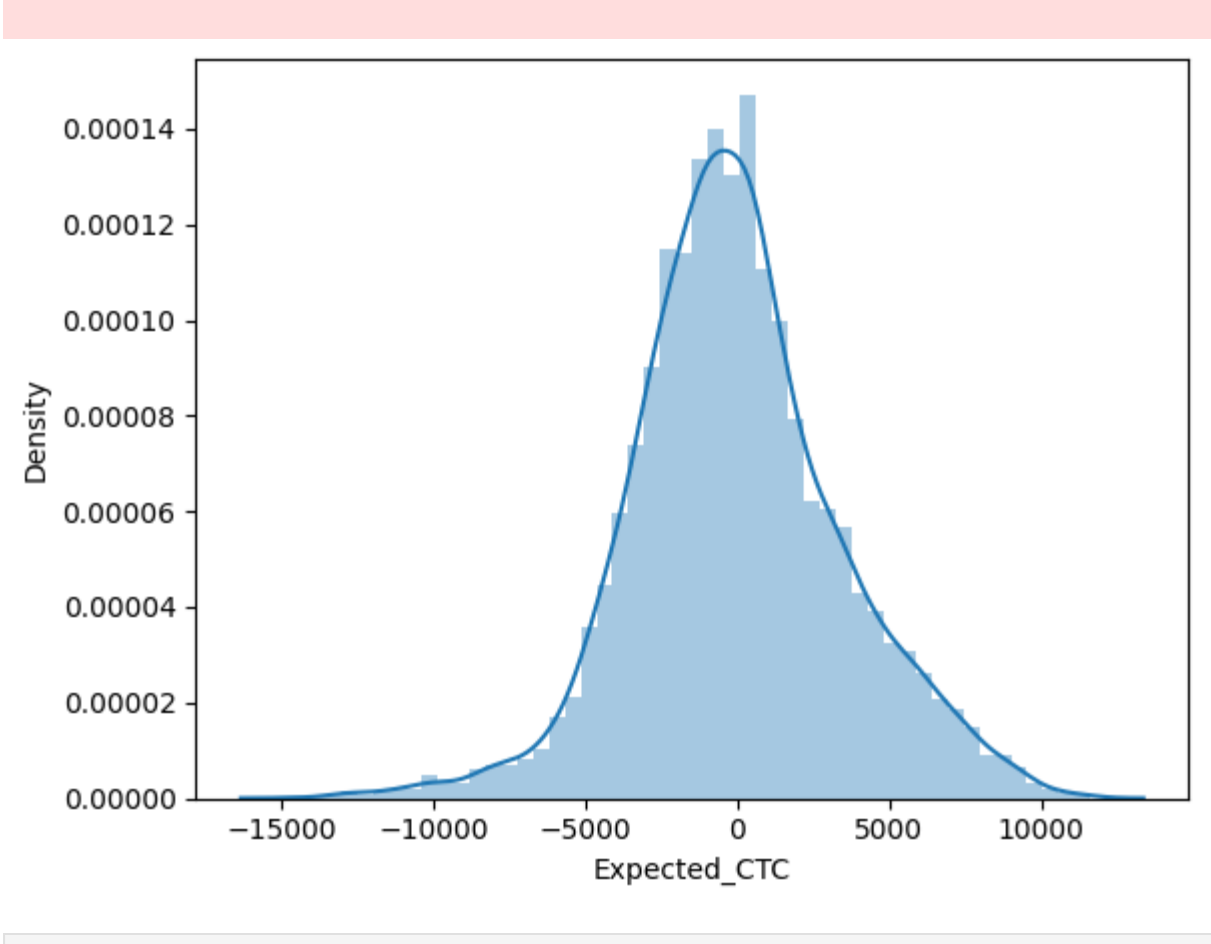
In [76]:
```
#Distplot()
sns.distplot(residual)
plt.show()
```

```
C:\Users\adity\anaconda3\envs\AntonioEnv\lib\site-packages\ipykernel_launcher.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```



In [77]:
```
#Find Skewness:
residual.skew()
```

Out[77]: 0.09524441015340776

In [78]:
```
# View shape and features of all datasets to be used
print("Number of test samples:", X_test.shape, "\nwith these features:\n", X_test.columns)
print("\nNumber of training samples:", X_train.shape, "\nwith these features:\n", X_train.columns)
print("\nNumber of test salaries:", Y_test.shape)
print("Number of training salaries:", Y_train.shape)
```

```
Number of test samples: (7500, 3)
with these features:
 Index(['Education', 'Total_Experience', 'Organization'], dtype='object')

Number of training samples: (17500, 3)
with these features:
 Index(['Education', 'Total_Experience', 'Organization'], dtype='object')

Number of test salaries: (7500,)
Number of training salaries: (17500,)
```

In [98]:
```
from sklearn.ensemble import RandomForestRegressor

#Create a Gaussian Classifier
rf=RandomForestRegressor(n_estimators=10)

#Train the model using the training sets y_pred=clf.predict(X_test)
rf.fit(X_train,Y_train)

y_pred_1=rf.predict(X_test)
```

In [99]:
```
y_pred_1
```

Out[99]:
```
array([ 7186.07833052, 16854.48392496, 16413.1502381 , ...,
       13823.23035408, 15218.00934343,  2914.50614316])
```

In [100...
```
#compre actual vs predicted o/p
dict={'Actual Output':Y_test,'Predicted output':y_pred_1}
#convert dictionary into dataframe
df_4=pd.DataFrame(dict)
df_4
```

Out[100...

	Actual Output	Predicted output
21492	5693	7186.078331
9488	9670	16854.483925
16933	16669	16413.150238
12604	2244	2286.250714
8222	1345	3780.907713
...	...	...
13410	6713	8348.215238
13158	21196	15248.207595
3552	13259	13823.230354
23203	21492	15218.009343
2410	3029	2914.506143

7500 rows × 2 columns

In [101...
```
#Mean Squared Error 1/n*sum(Y-Y-pred)**2
from sklearn.metrics import mean_squared_error
print("MSE: ",mean_squared_error(Y_test,y_pred_1))
```

```
MSE:  7188150.544894743
```

In [102...
```
mse=mean_squared_error(Y_test,y_pred_1)
rmse=np.sqrt(mse)
print("Root Mean Sqaured Error : ",rmse)
```

```
Root Mean Sqaured Error :  2681.0726481941406
```

In [103...
```
#R2--Score
#Check model is perfect or not
from sklearn.metrics import r2_score
print("R2-score : ",r2_score(Y_test,y_pred_1))
```

```
R2-score :  0.8619732816797295
```

In [104...
```
#Find Error or Residue
residual=Y_test-y_pred_1
print(residual.head())
```

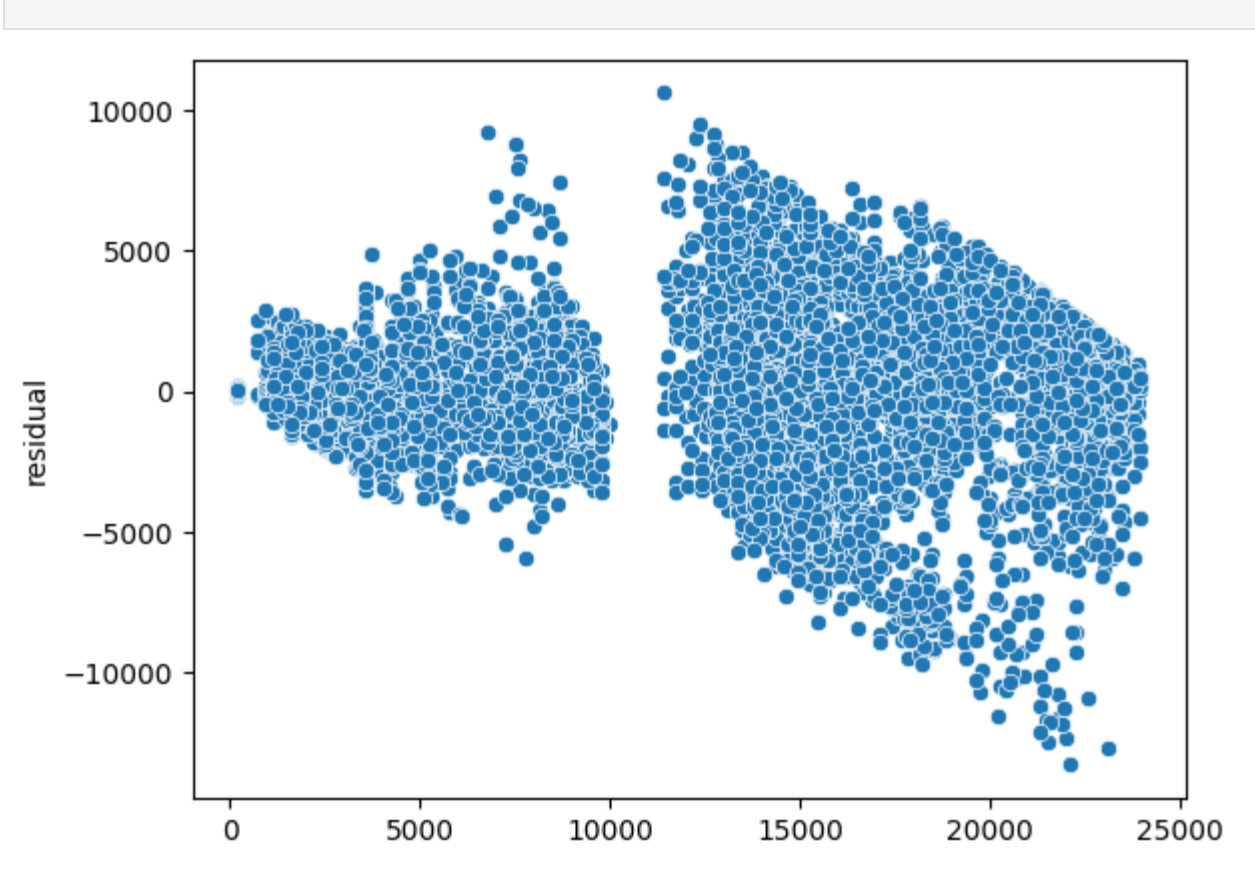
```
21492   -1493.078331
9488    -7184.483925
16933     255.849762
12604     -42.250714
8222    -2435.907713
Name: Expected_CTC, dtype: float64
```

In [106...
```
#Create a Dictionary:
dic={'Y_pred' :y_pred_1,'residual' : residual}
df_5=pd.DataFrame(dic)
df_5.head()
```

Out[106...

	Y_pred	residual
21492	7186.078331	-1493.078331
9488	16854.483925	-7184.483925
16933	16413.150238	255.849762
12604	2286.250714	-42.250714
8222	3780.907713	-2435.907713

In [107...
```
#Visualise scatterplot between Y_pred & residual means
#here input x=Y_pred & output y=residual because residual depends on Y_pred value.
sns.scatterplot(data=df_5,x=y_pred_1,y='residual')
plt.show()
```



In [108…

```
sns.distplot(residual)
plt.show()
```

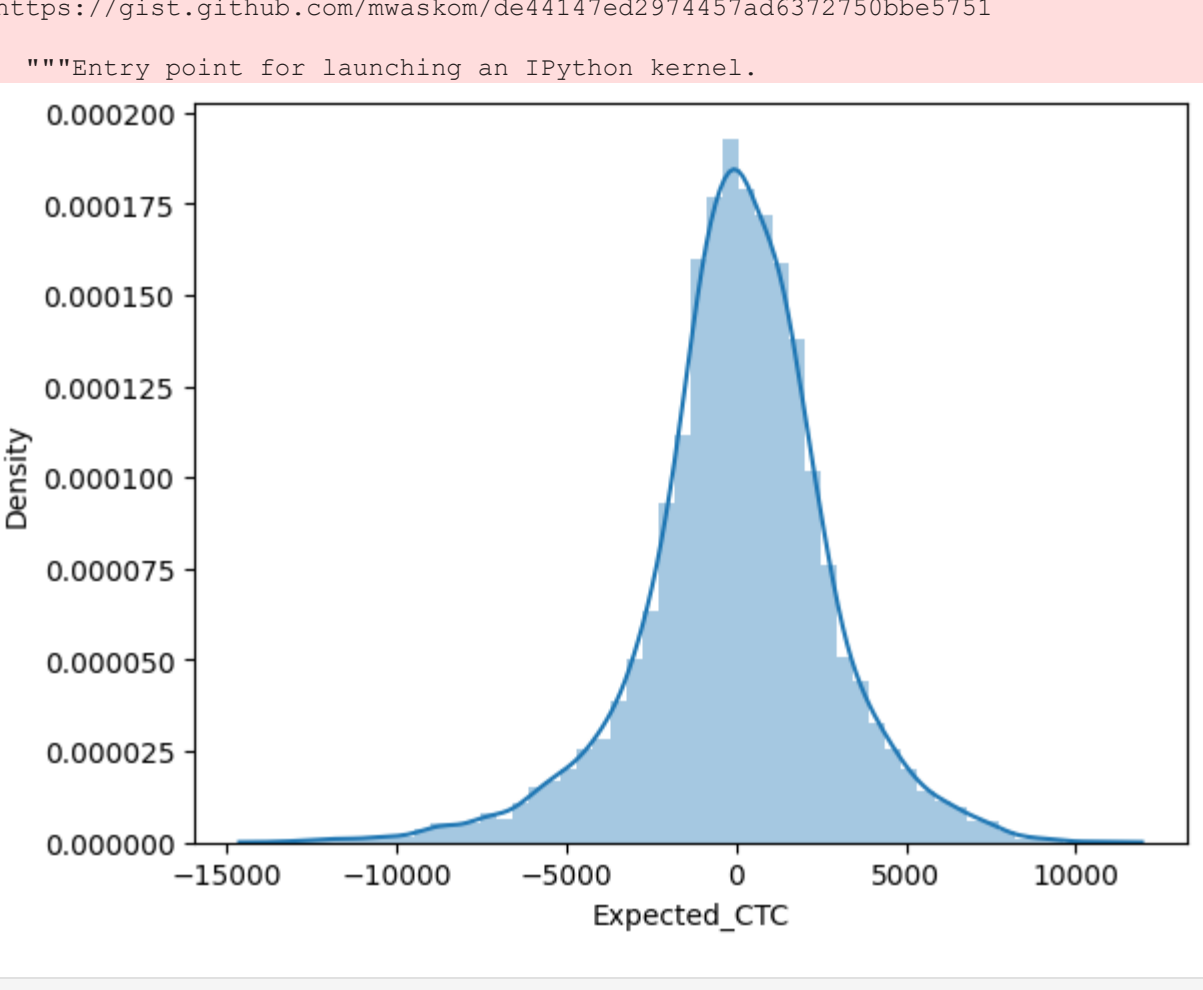
```
C:\Users\adity\anaconda3\envs\AntonioEnv\lib\site-packages\ipykernel_launcher.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  """Entry point for launching an IPython kernel.
```



In [ ]:

In [82]:

```
from sklearn.svm import SVR

sv = SVR(kernel='rbf')
sv.fit(X_train_scaled,Y_train)

y_pred = sv.predict(X_test_scaled)
```

In [83]:

```
y_pred
```

Out[83]:

```
array([11520.3204228 , 12339.63402108, 14078.99246861, ...,
       13338.63762989, 13013.88038503, 11113.47932917])
```

In [85]:

```
#compre actual vs predicted o/p
dict={'Actual Output':Y_test,'Predicted output':y_pred}
#convert dictionary into dataframe
df_2=pd.DataFrame(dict)
df_2
```

Out[85]:

	Actual Output	Predicted output
21492	5693	11520.320423
9488	9670	12339.634021
16933	16669	14078.992469
12604	2244	10570.658269
8222	1345	11027.742936
...	...	...
13410	6713	11158.703169
13158	21196	12110.788250
3552	13259	13338.637630
23203	21492	13013.880385
2410	3029	11113.479329

7500 rows × 2 columns

In [86]:

```
#Mean Squared Error 1/n*sum(Y-Y-pred)**2
from sklearn.metrics import mean_squared_error
print("MSE: ",mean_squared_error(Y_test,y_pred))
```

```
MSE:  36253766.18287494
```

In [87]:

```
mse=mean_squared_error(Y_test,y_pred)
rmse=np.sqrt(mse)
print("Root Mean Sqaured Error : ",rmse)
```

```
Root Mean Sqaured Error :  6021.110045736994
```

In [88]:

```
#R2--Score
#Check model is perfect or not
from sklearn.metrics import r2_score
print("R2-score : ",r2_score(Y_test,y_pred))
```

```
R2-score :  0.30385593043448134
```

In [89]:

```
#Find Error or Residue
residual=Y_test-y_pred
print(residual.head())
```

```
21492   -5827.320423
9488    -2669.634021
16933    2590.007531
12604   -8326.658269
8222    -9682.742936
Name: Expected_CTC, dtype: float64
```

In [90]:

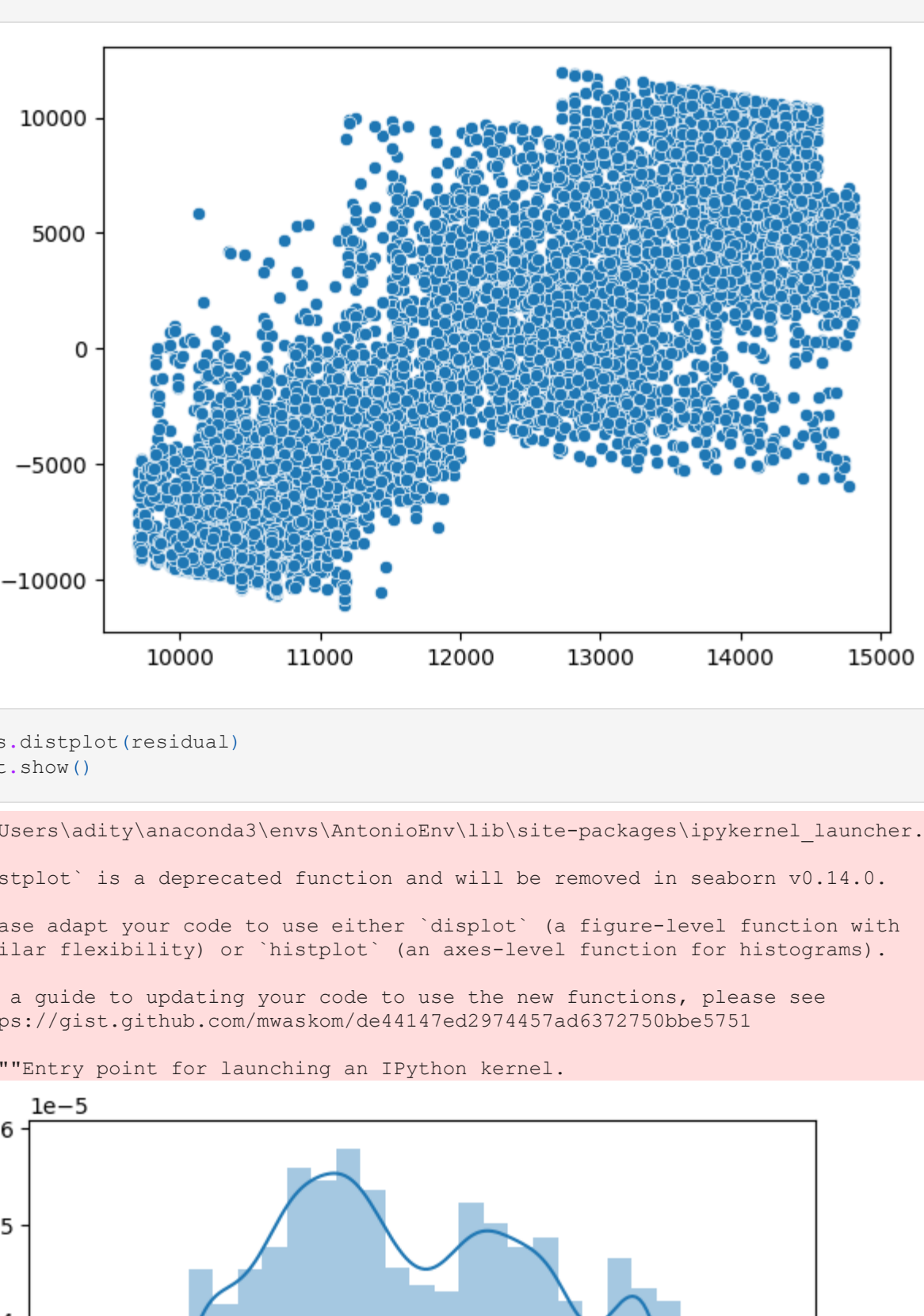
```
#Create a Dictionary:
dic={'Y_pred' :y_pred,'residual' : residual}
df_3=pd.DataFrame(dic)
df_3.head()
```

Out[90]:

	Y_pred	residual
21492	11520.320423	-5827.320423
9488	12339.634021	-2669.634021
16933	14078.992469	2590.007531
12604	10570.658269	-8326.658269
8222	11027.742936	-9682.742936

In [94]:

```
#Visualise scatterplot between Y_pred & residual means
#here input x=Y_pred & output y=residual because residual depends on Y_pred value.
sns.scatterplot(data=df_3,x=y_pred,y='residual')
plt.show()
```



In [95]:

```
sns.distplot(residual)
plt.show()
```

```
C:\Users\adity\anaconda3\envs\AntonioEnv\lib\site-packages\ipykernel_launcher.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  """Entry point for launching an IPython kernel.
```


In [ ]: